

Engineering Take Home Assignment – Front End Engineer

Task

Build a performant, real-time web application using React, Vue, or a modern TypeScript framework that connects to Polymarket's Gamma REST API and CLOB WebSocket feed. The dashboard allows users to select an active NFL game event and view real-time market updates for associated game outcome (moneyline) and totals contracts in a clean interface.

Constraints

- **Time Limit:** We expect this assignment to take no more than 2 to 3 hours.
- **APIs & Streaming:** Must connect directly to Polymarket's public Gamma REST API and CLOB WebSocket feed without relying on any custom backend proxies or intermediate servers.
- **Tech Stack:** Must be built using React, Vue, Svelte, or a modern TypeScript framework.
- **AI Tool Usage:** If you use any AI tool to help you perform this task, please indicate which model(s) you used and how you used them.

Guidelines

1. Event Selection

- Fetch NFL events from Polymarket's API, paginating with `offset` (the API caps each response at 100 events regardless of the `limit` value requested — a full week's slate requires walking several pages): `GET https://gamma-api.polymarket.com/events?tag_slug=nfl&active=true&clob=false&limit=100&offset=0`
- **`tag_slug=nfl` alone returns much more than needed** — including futures, props, entertainment markets, and separate sub-event objects for halves, quarters, and player stats.
- Filter to **one event per actual game** by matching the slug against the pattern `^nfl-[a-z]{2,4}-[a-z]{2,4}-\d{4}-\d{2}-\d{2}$` (team codes + kickoff date, nothing after — e.g. `nfl-kc-mia-2026-09-27`, title "Chiefs vs. Dolphins").
- Each matched event contains a nested `markets[]` array with **every** market for that game — moneyline, spread, dozens of alternate total lines, team totals,

quarter/half lines, and 250+ player props. **Scope for this assignment: Only the game outcome (moneyline) market plus every full-game total (over/under) line.**

- **Moneyline:** the one market whose `question` exactly equals the event's `title` (e.g. "Chiefs vs. Dolphins"). 2 outcomes.
- **Totals (full game, every line):** Polymarket lists 20+ alternate O/U lines per game ("Chiefs vs. Dolphins: O/U 41.5", ...O/U 43.5", etc.) — include **all** of them, not just one. Filter to markets matching `"{event.title}: O/U {number}"` exactly (no team-name prefix, no 1H/2H/1Q-4Q suffix). Each matched line has 2 outcomes (Over / Under).
- Each of these outcomes is backed by its own `clobTokenIds` (asset ID) used later for WebSocket subscriptions.

2. Live Market Table

Render a live-updating table displaying the game outcome and totals markets (one row per outcome/token) with the following columns:

Column	Source
Outcome	<code>market.question + outcome label</code>
Best Bid Price	The highest price in <code>bids[]</code> from the initial <code>book</code> snapshot, then <code>price_change.best_bid</code> on every subsequent delta for that asset
Best Ask Price	The lowest price in <code>asks[]</code> from the initial <code>book</code> snapshot, then <code>price_change.best_ask</code> on every subsequent delta for that asset
Last Traded Price	seeded from the <code>book</code> snapshot's own <code>last_trade_price</code> field, then updated on subsequent <code>last_trade_price</code> events (<code>price</code> field)
Bid-Ask Spread	Derived: <code>Best Ask Price - Best Bid Price</code>

3. Real-Time WebSocket Streaming

Connect to Polymarket's public market WebSocket:

None

```
wss://ws-subscriptions-clob.polymarket.com/ws/market
```

Subscribe by sending the token IDs (`assets_ids`) for every outcome in the selected event:

None

```
{
  "assets_ids": ["<token_id_1>", "<token_id_2>", "..."],
  "type": "market",
}
```

```
"initial_dump": true
}
```

Handle these message types from the feed:

- `book` — full order book snapshot per asset. Only needed once per asset, to read highest price in `bids[]` and the lowest price in `asks[]` as the starting best bid/ask and to seed `last_trade_price/tick_size` — you do not need to maintain a full order book after this, since size is out of scope.
- `price_change` — incremental updates (`price_changes[]`, one item per changed asset); read that item's own `best_bid/best_ask` fields directly as the new best bid/ask price for that asset — no order-book reconstruction needed.
- `last_trade_price` — trade execution updates (`price, size, side`) that arrive *after* the initial book snapshot
- `tick_size_change` — minimum tick size updates that arrive *after* the initial book snapshot, needed to correctly round/display prices

Send a literal `"PING"` text frame every 10 seconds to keep the connection alive.

- Update table state efficiently as messages stream in without triggering full-table re-renders.
- **Visual Flash Indicators:** Flashing visual highlight on changing cells (e.g. green flash for price increase, red for price decrease) that auto-fades after 500ms.
- **Subscription Management:** When switching events, unsubscribe from the previous event's token IDs and subscribe to the new event's token IDs on the same connection (no need to reconnect the whole socket):

None

```
{ "operation": "unsubscribe", "assets_ids": ["<old_token_id>",  
"..."] }
```